Named Shape Stores

Identity, Navigation and Scoped Unification

Mettapedia Working Notes

31 October 2026

Abstract

A name can remain usable when its term is represented by a record, a skeleton with named holes, or a different cut of the same syntax. This note develops the logical contracts for that separation in Lean. Canonical admission reuses existing names despite routing changes; checked relocation preserves every decoded name. Recursive shape bodies are constructed with a decreasing subterm rank. A store-aware cursor agrees with navigation on the expanded tree, while a scoped constraint compiler has exactly the solutions of the expanded first-order unification problem. Acyclic reference collection and a reachable-name size bound complete the representation model. The results are semantic oracles, not verification of C accessors, physical allocation, concurrency, or language-level reflection.

1 Three identities and an explicit observation

The term carrier T fixes the equality preserved by the codecs. In the shape instance it is finite binary constructor syntax with labeled leaves. Open syntax distinguishes constant labels from variable markers. Equal payloads do not identify distinct stored occurrences: domain-qualified occurrence identity is treated separately in `PersistentOccurrenceSegments`. Nor is a content name automatically a quoted process. Relating these codecs to quotation and dereferencing in a language requires that language's structural congruence and observation rules.

The constructions here preserve exact decoded syntax. A semantic quotient, alpha-normalization algorithm, or task-relative equivalence may supply a different carrier, but its soundness and completeness cannot be inferred from the codec laws. Similarly, equality of canonical codes says nothing by itself about their physical comparison cost.

2 Canonical homes, admission and movement

A codec has an encoder $e : T \to C$ and a partial decoder $d : C \to \mathrm{Option}(T)$ with

$$d(e(t)) = \mathrm{some}(t).$$

Consequently e is injective. This does not make d injective on all raw codes: noncanonical representations may be aliases. For a family of stores $s \in S$ and a chosen home $h : T \to S$, define

$$N = \sum_{s \in S} C_s, \qquad \mathrm{name}_h(t) = (h(t), e_{h(t)}(t)).$$

The round trip proves $\mathrm{name}_h(t) = \mathrm{name}_h(u) \iff t = u$. Decoding is injective on this canonical image, not on the whole disjoint union.

Stores need not encode every term. `CodecOn` restricts the encoder to an admission domain, and `AdmittedNaming` proves that the selected home admits its term. The concrete shape-table instance chooses the skeleton returned by a cut as the store and uses the ordered list of hole values as the row code. Its round trip is the existing `ShapeStore.Cut.sec` law.

A routing change is not an identity migration. If $h(t) \neq h'(t)$, the two store-qualified names differ, although both decode to t. Retaining old stores preserves old reads, but admitting t afresh under h' creates an alias. `CanonicalNameAdmission.intern` prevents this by consulting the term-keyed catalog first. A present valid entry wins over every new proposal; a stale entry is refused rather than silently renamed. Successful admission preserves old entries and catalog soundness. Retry under any later proposal policy returns exactly the published name.

For moving placements, a directory maps stable names to physical codes. `NamesOverStores.relocate` accepts a target only when it decodes to the existing live value. It preserves the complete name-to-value function. This law composes over any finite accepted relocation sequence. It does not prove atomic native publication or safe recycling of a physical slot.

3 Recursive holes and finite expansion

A named shape expression is

$$E ::= \mathrm{atom}(a) \mid \mathrm{node}(E, E) \mid \mathrm{ref}(n).$$

The skeleton forgets reference contents, replacing each reference with a hole. The ordered reference list retains repeated occurrences. For a value environment $v : N \to T$, plugging satisfies the contextual framing law

$$\mathrm{fill}(\mathrm{skel}(E), \mathrm{map}(v, \mathrm{refs}(E)) +\!\!+ r) = \mathrm{some}(\mathrm{plug}(v, E), r).$$

Replacing references by further expressions is associative, has reference insertion as its identity, and commutes with plugging and partial evaluation. Thus nested holes compose without introducing a separate substitution environment per layer.

For a partial heap $H : N \to \mathrm{Option}(E)$, bounded expansion dereferences one level per unit of depth. Successful expansion is monotone in that allowance and yields a unique value. If every referenced child is present and has smaller rank, an entry n expands within $\rho(n) + 1$ reference levels.

The representation construction earns that rank. `quoteBody` retains the root constructor and quotes only selected proper subterms. Starting from a naming round trip, `NamedStoreModel.bodies` creates these bodies for every decoded name. Term size strictly decreases along every outgoing reference. `bodies_complete` proves expansion back to the original term, and `recut_preserves_name` proves that any two cut policies retain the same decoded value at the same name. The construction is a reference representation; no efficient native admission algorithm is implied.

4 Navigation and shared-size bounds

`NamedShapeCursor.seek` follows a Boolean left/right path by exposing the current body only. It does not call whole-term expansion. Under the constructed body model, its focus has exactly the value selected by the independently defined tree operation `subterm`. Invalid paths are refused on both sides. Path concatenation agrees with sequential navigation. Zipper frames retain the sibling; downward and upward motion preserve the reconstructed meaning. Moving up may

produce an inline constructor in place of the original raw name, so a raw-encoding round trip is deliberately not claimed.

An instrumented expansion records each dereference occurrence. Projecting away this trace recovers ordinary expansion. If every stored body contributes at least one constructor, then

$$|\,\text{distinctVisitedNames}\,| \;\le\; |\,\text{dereferenceOccurrences}\,| \;\le\; |\,\text{expandedTerm}\,|.$$

The trace contains its root and is closed under the stored reference edges. Consequently every finite set of reachable names obeys the same bound. `constructed_reachable_bound` supplies the constructor condition and successful expansion from the body construction itself.

This is a bound on logical name entries, not bytes, all allocated names, shared skeleton metadata, or caches. Alias-only chains violate the constructor condition: two named entries may expand to a single atom. The negative control records precisely this counterexample.

5 Scoped first-order unification

Storage labels for variables are data, but when a row participates in unification the labels denote logic variables. A row activation receives one scope, shared by every variable occurrence in its skeleton and holes. The query has its own scope. Repeated variable markers remain equal; independent row activations can use distinct scopes.

`ShapeScopedUnification.constraints` walks aligned query and shape constructors, emitting a joint system of equations. Other positions produce equations involving the appropriate subterm. For every substitution θ,

$$\theta \models \text{constraints}(p, E) \quad \Longleftrightarrow \quad \theta(\text{encode}_q(p)) = \theta(\text{encode}_r(\text{plug}(v, E))).$$

This is equality of complete solution sets, not merely agreement of one test answer. Reusing the existing total first-order unifier yields soundness, completeness and a most-general unifier. Staged and fully expanded answers factor through one another; they need not be identical substitution functions. Re-cutting while preserving the expanded value preserves all solutions.

The reference fallback still materializes a subterm when a query variable faces constructor structure. The result is therefore an oracle for a future fully name-based solver, not a claim that all unification now avoids decoding. Alpha-normalization, native binding-store refinement, rational trees and higher-order unification are not supplied here.

6 Cross-store holders and collection

Let A be the finite allocated-name set, R the list of external root slots, and $\text{refs}(p)$ the outgoing slots of a stored name. The census is

$$c_A(n) = \text{count}_R(n) + \sum_{p \in A} \text{count}_{\text{refs}(p)}(n).$$

Repeated slots are counted separately. Store boundaries do not interrupt counting; counts over disjoint store inventories add exactly.

One pruning round retains names with positive count. Every round preserves rooted reachability and reference closure. After at most $|A|$ rounds the set is stable. With strictly decreasing reference ranks, every remaining name is rooted or has a finite predecessor chain to a root. Thus the final set is exactly the reachable names. Without acyclicity, an unrooted cycle survives every zero-count round. Tracing or separate cycle handling remains necessary in that case.

These laws concern a sequential census-and-prune algorithm. They do not prove the runtime's incremental counter updates, complete root discovery, concurrent collection, weak-catalog handling, or an allocator's stale-name policy. In particular, a content catalog must not accidentally become an undeclared strong root for every term ever admitted.

7 Qualification and integration boundary

The eight mathematical modules live under `Mettapedia/Machines/`: `NamesOverStores`, `CanonicalNameAdmission`, `NamedShapeGraph`, `ShapeScopedUnification`, `NamedStoreHolders`, `NamedStoreModel`, `NamedShapeCursor` and `NamedDAGSize`. `NamedStoreAudit` inventories their module-owned mathematical bodies, including private and generated helpers. The permitted transitive axioms are `propext`, `Classical.choice` and `Quot.sound`; runtime compiler counterparts are listed separately. No admissions or new axioms are used.

Controls include two stores decoding the same term to unequal raw names, changed homes, stale catalog entries, overwritten placements, missing names, cyclic references, repeated shared edges, invalid paths, per-hole freshening, rigid treatment of variable markers, the occurs check, omitted cross-store holders, and alias-only size inflation.

Native integration must establish the read/exposure relation, one scope per row activation, canonical admission under routing changes, and complete lifetime roots. Performance and memory claims require runtime measurements. The remaining broader theory includes restricted transducer compilation and CTL fixed points, physical decision-diagram canonicity, categorical unifier properties, higher-derivative identifications, and workload-dependent cut optimization. None is inferred from the results in this note.